

2022-9-1

文档编号		版本	1.0
文档名称	低速总线虚拟化配置		
项目来源			

低速总线虚拟化配置

编写：日期： 2025-10-25

检查：日期：

审核：日期：

批准：日期：

目录

一、SPI3

 1、 SPI 虚拟化配置方法3

 2、 SPI passthrough 配置方法4

二、 I2C4

 1、 I2C 虚拟化配置方法4

 2、 I2C passthrough 配置方法6

三、 UART7

 1、 UART 虚拟化配置方法7

 2、 UART passthrough 配置方法8

低速总线虚拟化配置

一、SPI

包括 76 有 7 路 SPI

1、SPI 虚拟化配置方法

Yocto 配置

```
CONFIG_DEVICE_MODULES_VHOST_SPI=m
```

Android 配置

```
CONFIG_VIRTIO_SPI=m
```

//后端配置

//在 spi 总线上添加从设备

```
&spi1 {
    pinctrl-names = "default";
    pinctrl-0 = <&ak7709_codec_spi>;
    /*pinctrl-1 = <&ak7709_codec_pdn>;*/
    status = "okay";
    ak7709: ak7709@0 {
        status = "okay";
        compatible = "akm,ak7709";
        #sound-dai-cells = <0>;
        reg = <0>;
        spi-max-frequency = <1000000>;
        /*ak7709, pdn-gpio = <&pio 147 0>;*/
    };
};
```

};

//添加 pinctrl 以设置 SPI pinmux

```
&pio {
    ak7709_codec_spi: spimode-default {
        pins-cmd-dat {
            pinmux = <PINMUX_GPIO53__FUNC_SPI1_M0>,
                <PINMUX_GPIO52__FUNC_SPI1_MI>,
                <PINMUX_GPIO51__FUNC_SPI1_CSB>,
                <PINMUX_GPIO50__FUNC_SPI1_CLK>;
        };
    };
};
```

//使能虚拟化

```
vhost_spi: vhost-spi {
    compatible = "grt,vhost-spi";
};
```

//前端配置

```
spi1_virtio: spi1-virtio {
    compatible = "mediatek,virtio-spi";
    id = <1>;
    status = "ok";
};
```

2、SPI passthrough 配置方法

理论当 yocto spi adapter 正常情况下，以及 Android spi adapter 驱动已经加载的情况下，只需要把 spi 的中断号配置给 Android。以 spi1 为例

1、yocto disable

```
&spi1 {
    status = "disabled";
}
```

2、android enable

```
&spi1 {
    status = "ok";
}
```

3、spi1 中断号配置给 Android

```
yocto/prebuilt/hypervisor/grt_mt8676/uos_alps_pv8676_xx.lua
+uos_config:setVgicIrq(275)
```

spi 中断号=设备数 interrupts243+32 = 275

二、I2C

76 总共有 0~13 14 路 i2c，速率最高可配 3.4Mb/s，默认所有 i2c 控制器的中断都配置到 sos。

1、I2C 虚拟化配置方法

先了解下前后端驱动路径，如下：

1、前端驱动

vendor/mediatek/kernel_modules/virt

2, 后端驱动

1) thyp sdk 向 uos 映射 pci 设备

thyp-sdk/vmm/nbl_vmm/

2) vhost 驱动

src/kernel/modules/mt8676/virt/grt

i2c vhost 方式配置

1.1 Android

1, 移植 i2c-virtio 驱动及 build 配置

virtio 驱动放到 vendor/mediatek/kernel_modules/virt 路径下。

并添加必要 BUILD.bazel, Kbuild, Makefile build 脚本。

GKI 还需要在 bazel 文件添加如下内容:

```
kernel/kleaf/mgk_64_k61.bzl
+mgk_64_k61_kleaf_modules.append("//vendor/mediatek/kernel_modules/virt/i2c_virtio:i2c-virtio")
kernel/build/bazel_mgk_rules//mgk.bzl
+="//vendor/mediatek/kernel_modules/virt/i2c_virtio:i2c-virtio.{.}.{.}.format(name, "6.1", build),
```

若系统未生成/dev/i2c-X 节点, 需要在 defconfig 添加

```
CONFIG_I2C_CHARDEV=y
```

2, dts 配置

如 i21 使用虚拟化, id 需要为 1

```
i2c1_virtio: i2c1-virtio {
compatible = "mediatek,virtio-i2c";
id = <1>;
status = "disabled";
};
3, ko_order_table.csv
device/mediateksample/auto8676p1_64_bsp_vm/ko_order_table.csv
+i2c-virtio.ko,./vendor/mediatek/kernel_modules/virt/i2c_virtio/i2c-virtio.ko,vendor,Y,N,user/userdebug/eng
```

1.2, uos yocto

1, 移植 i2c vhost 驱动及 build 配置

src/kernel/modules/mt8676/virt/grt

并添加必要 BUILD.bazel, Kbuild, Makefile build 脚本。

2, dts 配置

```
meta/meta-mediatek-mt8676-hyp//recipes-kernel/linux/files/auto8676p1_64_hyp.dts
&i2c_vhost {
compatible = "grt,vhost-i2c";
status = "okay";
};
3, ko_order_table.csv
meta/meta-mediatek-mt8676-hyp//recipes-kernel/linux/ko_order_table/auto8676p1_64_hyp/ko_order_table.csv
+i2c_vhost.ko,./vendor/mediatek/kernel_modules/virt/grt/i2c_vhost/i2c_vhost.ko,vendor,Y,Y,user/userdebug/eng
```

1.3, thyp

1, thyp 添加 i2c 支持

grt_mt8676/thyp-sdk/vmm/nbl_vmm/BUILD.bazel

```
cc_binary(
```

```
name = "nbl_vmm_mix",
```

```
srcs = [":nbl_vmm_srcs"],
```

```
deps = [
```

```
":vmmso",
```

```
":libnbl",
```

```
":net",
```

```
+ ":i2c",
```

```
"@third_party//rapidjson",
```

```
"@third_party//prebuilts/libluajit",
```

```
],
```

WITH_VIRTIO_I2C 将会被定义

grt/thyp-sdk/vmm/nbl_vmm/[main.cc](#)

```
#ifdef WITH_VIRTIO_I2C
```

```
#include "nbl_vmm/devices/i2c.h"
```

```
#endif
```

```
#ifdef WITH_VIRTIO_I2C
```

```
if (config.is_vi2c_enabled()) {
```

```
status = machina::attach_vhost_i2c(instance.get(), config);
```

```
CHECK(status == ZX_OK);
```

```
LOG(ERROR) << "i2c status " << status; <="" div=""> </status>;
```

```
}
```

```
#endif
```

将会调用 grt/thyp-sdk/vmm/nbl_vmm/devices/[i2c.cc](#) 中 attach_vhost_i2c, 完成 i2c vhost 方式注册。

2、 I2C passthrough 配置方法

理论当 yocto i2c adapter 正常情况下, 只需要把 i2c 的中断号配置给 Android。

以 i2c10 直通 Android 为例

1.1 Android

1, i2c-mt65xx ko

```
diff --git a/ko_order_table.csv b/ko_order_table.csv
```

```
+i2c-mt65xx.ko,./kernel_device_modules-6.1/drivers/i2c/busses/i2c-mt65xx.ko,vendor,Y,N,user/userdebug/eng
```

1.2 yocto

1, i2c10 gpio 配置

```
diff --git a/dws/mt6897/auto8676p1_64_hyp.dws b/dws/mt6897/auto8676p1_64_hyp.dws
```

```
+      <gpio157>
+
+      <eint_mode>false</eint_mode>
+
+      <def_mode>1</def_mode>
+
+      <inpull_en>true</inpull_en>
+
+      <inpull_selhigh>true</inpull_selhigh>
+
+      <def_dir>IN</def_dir>
+
+      <out_high>false</out_high>
+
+      <smt>true</smt>
+
+      <ies>true</ies>
+
+    </gpio157>
+
+    <gpio158>
+
+      <eint_mode>false</eint_mode>
+
+      <def_mode>1</def_mode>
+
+      <inpull_en>true</inpull_en>
+
+      <inpull_selhigh>true</inpull_selhigh>
+
+      <def_dir>IN</def_dir>
+
+      <out_high>false</out_high>
+
+      <smt>true</smt>
+
+      <ies>true</ies>
+
+    </gpio158>
```

1.3 thyp-sdk

i2c10 中断号配置给 Android, lua 文件配置需要 根据 dts 中配置的中断号加偏移, +32

i2c10 dts 中断配置如下:

```
interrupts = <GIC_SPI 196 IRQ_TYPE_LEVEL_HIGH 0>;
```

uos_alps_pv8676.lua 则需要 $196+32 = 228$

```
+uos_config:setVgiclrq(228)
```

编译到版本

```
cp uos_alps_pv8676.lua yocto/prebuilt/hypervisor/grt_mt8676/uos_alps_pv8676_mix.lua
```

三、UART

76 总共有 0~3 4 路 uart, 速率最高可配 3Mb/s, 默认所有 uart 控制器的中断都配置到 sos。

1、UART 虚拟化配置方法

采用 virtio-console 方式

1, front driver

kernel/drivers/char/virtio_console.c

2, backend driver

kernel/modules/mt8676/virt/grt/vhost_uart/uart.c

3, 测试前端生成的 uart 虚拟节点

```
find . -name vport*
```

获取 uart 数据

```
cat /dev/vport*
```

uart 发送数据

```
echo 123 > /dev/vport*
```

2、UART passthrough 配置方法

理论当 yocto 和 android 端 uart 驱动加载正常情况下，只需要把 uart 的中断号配置给 Android，uart 便直通到 Android。

1, Android

uart dts 配置

```
--- a/arch/arm64/boot/dts/mediatek/auto8676p1_64_bsp_vm.dts
+++ b/arch/arm64/boot/dts/mediatek/auto8676p1_64_bsp_vm.dts

+&pio {

+    uart2_pins_default: uart2-pins-default {

+        pins-rx {

+            pinmux = <PINMUX_GPIO117_FUNC_URXD2>;

+            input-enable;

+            bias-pull-up;

+        };

+        pins-tx {

+            pinmux = <PINMUX_GPIO121_FUNC_UTXD2>;

+        };

+        pins-rtx {

+            pinmux = <PINMUX_GPIO162_FUNC_URTS2>;

+            output-enable;

+        };

+        pins-cts {

+            pinmux = <PINMUX_GPIO161_FUNC_URTS2>;
```

```

+         input-enable;

+     };

+ };

+};

+

&uart2 {

-     status = "disabled";

+     pinctrl-names = "default";

+     pinctrl-0 = <&uart2_pins_default>;

+     status = "okay";

};

&uarthub {

-     status = "disabled";

+     status = "okay";

};

uart 驱动 ko 加入到 ko_order_table.csv

--- a/ko_order_table.csv

+++ b/ko_order_table.csv

+mtk-uart-apdma.ko,../kernel_device_modules-6.1/drivers/dma/mediatek/mtk-uart-apdma.ko,ramdisk,Y,Y,user/userdebug/eng

+uarthub_drv.ko,../kernel_device_modules-6.1/drivers/misc/mediatek/uarthub/uarthub_drv.ko,ramdisk,Y,Y,user/userdebug/eng

+8250_mtk.ko,../kernel_device_modules-6.1/drivers/tty/serial/8250/8250_mtk.ko,ramdisk,Y,Y,user/userdebug/eng


2.  yocto

disable uart2

--- a/recipes-kernel/linux/files/auto8676p1_64_hyp_sos.dts

+++ b/recipes-kernel/linux/files/auto8676p1_64_hyp_sos.dts

@@ -393,7 +393,7 @@

&uart2 {

    pinctrl-names = "default";

    pinctrl-0 = <&uart2_pins_default>;

-     status = "okay";

+     status = "disabled";

};

```

uart2 gpio 配置成串口 mode

--- a/dws/mt6897/auto8676p1_64_hyp.dws

+++ b/dws/mt6897/auto8676p1_64_hyp.dws

@@ -1,5 +1,5 @@

<?xml version="1.0" encoding="UTF-8"?>

-<!--dct_version="2.5" buid_sn="161102" dws_modification_time="11.12.2024.15:42:26"-->

+<!--dct_version="2.5" buid_sn="161102" dws_modification_time="11.13.2024.14:22:06"-->

<dct_cfg>

 <general chip="MT6897">

 <proj>Ponsot_Phone</proj>

@@ -1265,10 +1265,10 @@

 </gpio116>

 <gpio117>

 <eint_mode>>false</eint_mode>

- <def_mode>0</def_mode>

+ <def_mode>3</def_mode>

 <inpull_en>true</inpull_en>

- <inpull_selhigh>>false</inpull_selhigh>

- <def_dir>OUT</def_dir>

+ <inpull_selhigh>true</inpull_selhigh>

+ <def_dir>IN</def_dir>

 <out_high>>false</out_high>

 <smt>true</smt>

 <ies>true</ies>

@@ -1306,10 +1306,10 @@

 </gpio120>

 <gpio121>

 <eint_mode>>false</eint_mode>

- <def_mode>0</def_mode>

- <inpull_en>true</inpull_en>

+ <def_mode>3</def_mode>

+ <inpull_en>>false</inpull_en>

 <inpull_selhigh>>false</inpull_selhigh>

- <def_dir>OUT</def_dir>

```
+      <def_dir>IN</def_dir>

      <out_high>false</out_high>

      <smt>true</smt>

      <ies>true</ies>
```

3. thyp-sdk

uart2 控制器中断号 284 配置给 Android，DMA 中断 255，256 也要配置给 Android。

uos_alps_pv8676.lua

```
+uos_config:setVgicIrq(284)
```

```
+uos_config:setVgicIrq(255)
```

```
+uos_config:setVgicIrq(256)
```